Module 2: Behavioral Design Patterns

BDP: Behavioural design patterns focus on how objects communicate and interact with each other.

They define how responsibilities and algorithms are distributed among objects.

They help make communication between objects: Flexible; Maintainable; Loosely coupled; Easier to extend.

Major Behavioral Patterns:

1. Template Method Pattern
2. Chain of Responsibility Pattern
3. State Pattern
4. Command Pattern
5. Observer Pattern

Key idea: Behavioural patterns focus on object communication and behavior, rather than object creation or structural organization.

WHAT ARE BEHAVIOURAL DESIGN PATTERNS?

They define how responsibillities and algorithms are distributed among objects.

Behavioural design patterns focus on how objects **communicate** and **interact** with each other.

THEY HELP MAKE COMMUNICATION BETWEEN OBJECTS

PLEXIBLE
Easily adapt to changing requirements.

MAINTAINABLE
Changes in one part do not affect others.

LOOSELY COUPLED
Objects depend on each other minimally.

EASIER TO EXTEND
New behavior can be added with minimal changes.

KEY IDEA

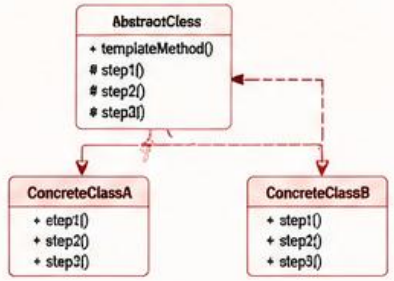
Behavioural patterns focus on object communication and behavior, rather than object creation or structural organization.

MAJOR BEHAVIOURAL PATTERNS (Examples)

1. TEMPLATE METHOD PATTERN

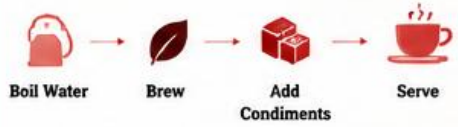
Defines the skeleton of an algorithm in a base class but lets subclasses override specific steps of the algorithm without changing its structure.

UML DIAGRAM



EXAMPLE

Making tea or coffee follows the same steps (boil water, brew, add condiments) but some steps vary.



JAVA CODE EXAMPLE

```
// Abstract Class
abstract class Beverage {
  // Template Method
  final void prepareRecipe() {
    boilWater();
    brew();
    addCondiments();
    serve();
  }
  void boilWater() {
    System.out.println("Boiling water");
  }
  abstract void brew();
  abstract void addCondiments();
  void serve() {
    System.out.println("Serving drink");
  }
}
```

```
// Concrete Class A
class Tea extends Beverage {
  void brew() {
    System.out.println("Steeping tea");
  }
  void addCondiments() {
    system.out.println("Adding lemon");
  }
```

```
// Concrete Class B
class Coffee extends Beverage {
  void brew() {
    System.out.println("Dripping coffee");
  }
  void addCondiments() {
    System.out.println("Adding sugar and milk");
  }
}
```

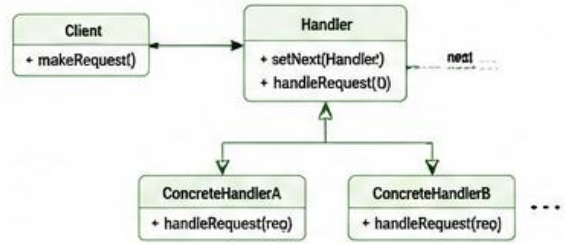
USE WHEN: You have an algorithm with fixed steps, but some steps vary.

BENEFITS: Code reuse, avoids duplication, easy to maintain.

2. CHAIN OP RESPONSIBILITY PATTERN

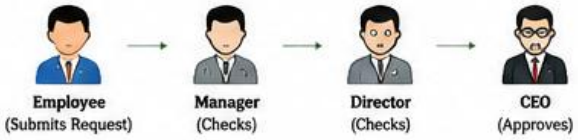
Passes a request along a chain of handlers until one of them handles the request.

UML DIAGRAM



EXAMPLE

A leave request is handled by Manager, then Director, then CEO if needed.



JAVA CODE EXAMPLE

```
// Handler Interface
abstract class Handler {
  protected Handler next;
  public void setNext(Handler next) {
    this.next = next;
  public abstract void handleRequest(int
    amount);
}
```

```
// Concrete Handlers
class Manager extends Handler {
  public void handleRequest(int amount) {
    if (amount == 1000) {
      System.out.println("Manager approved: " + amount);
    } else if (next != null) next.handleRequest(amount);
  }
```

```
class Director extends Handler {
  public void handleRequest(int amount) {
    if (amount == 5000) {
      System.out.println("Director approved: " + amount);
    } else if (next != null) next.handleRequest(amount);
  }
}
class CEO extends Handler {
  public void handleRequest(int amount) {
    if (amount = 5000) {
      System.out.println("CEO approved: " + amount);
    } else if (next != null) next.handleRequest(amount);
    }
  }
```

```
// Client
public class Client {
  public static void main(String[] args) {
    Handler director = new Manager();
    Handler ceo = new CEO();
    manager.setNext(director);
    manager.handleRequest(800);   // Manager approves
    manager.handleRequest(3000);  // Director approves
    manager.handleRequest(10000); // CEO approves
  }
```

USE WHEN: More than one object can handle a request, and the handler is not known in advance.

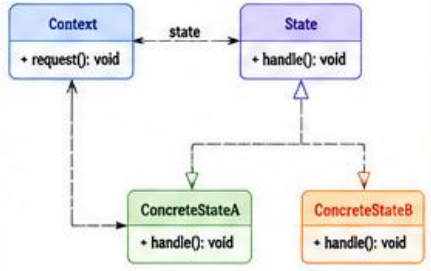
BENEFITS: Decouples sender and receiver, easy to add new handlers, follows OCP.

Key Idea: Behavioural patterns focus on object communication and behavior, rather than object creation or structural organization.

3. STATE PATTERN

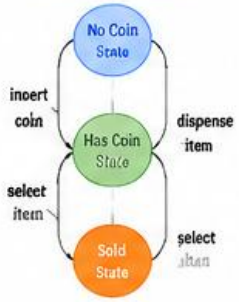
Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

UML DIAGRAM



EXAMPLE

Vending Machine



JAVA CODE EXAMPLE

```
// State Interface
interface State {
  void insertCoin();
  void selectItem();
  void dispense();
}
```

```
// Concrete State: NoCoinState
class NoCoinState implements State {
  private VendingMachine machine;
  public NoCoinState(VendingMachine m){
    this.machine = m;
  }
  public void insertCoin( {
    System.out.println("Coin inserted.");
    machine.setState(machine.getHasCoinState();
  } [];
  public void selectItem( {
    System.out.println("insert coin first.");
  }
  public void dispense( {
    System.out.println("insert coin first.");
  }
```

```
// Context
class VendingMachine {
  private State noCoinState;
  private State hasCeinState;
  private State soldState;
  private State state;
  }
  public VendingMachine( {
    noCoinState = new NoCoinState(this);
    hasCeinState = new HasCoinState(this);
    soldState    = new SoldState(this);
    state  = noCoinState;
  }
  public void setState(State s { state = s; }
  public State getHasCeinState( { return
  public void insertCoin( { state.insertCoin(); }
  public void selectItem( { state.selectItem(); }
  public void dispense( { state.dispense(); }
}
```

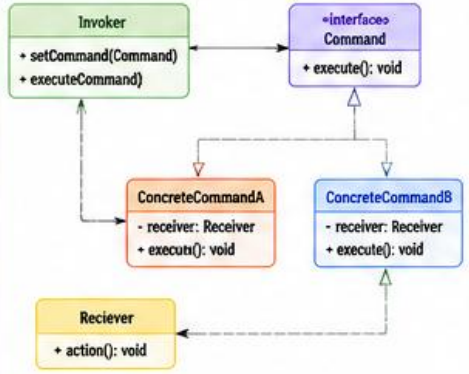
How it works: The context delegates the request to the current state object. When state changes, a different state object handles the requests.

Benefits: Eliminates large conditional statements, easy to add new states.

4. COMMAND PATTERN

Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undo operations.

UML DIAGRAM



EXAMPLE

Remote Control



Invoker (Press Button) — Receiver (TV)

- Button 1 → Turn On TV
- Button 2 → Turn Off TV
- Undo → Previous Command

JAVA CODE EXAMPLE

```
// Command Interface
interface Command {
  void execute();
  void undo();
}
// Concrete Command: TurnOn
class TurnOnCommand implements Command {
  private TV tv;
  private boolean wasOn;
  public TurnOnCommand(TV tv) {
    this. = tv; }
  public void execute( {
    wasOn = tv.isOn();
    tv.turnOn();
  }
  public void undo( {
    if(!wasOn) tv.turnOff();
}
```

```
// Receiver
class TV {
  private boolean on = false;

  public void turnOn( {
    on = true;
    System.out.println("TV is ON";
  }
  public void turnOff( {
    on = false;
    System.out.println("TV is OFF";
  }
  public boolean isOn( { return
    on; }
}
```

```
// Invoker
class RemoteControl {
  private Command command;
  private Command lastCommand;
  public void setCommand(Command cmd) {
    this.command = cmd;
    lastCommand = cmd;
  public void pressButton( {
    if(command != null){
      command.execute();
      lastCommand = command;
      lastCommand = command;
    }
  public void pressUndo( {
    if(lastCommand != null){
  }
}
```

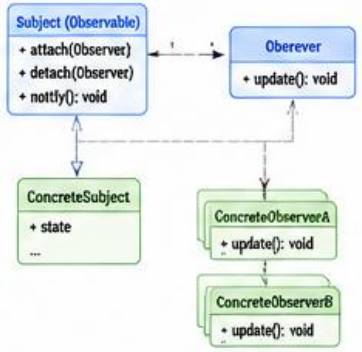
How it works: The invoker sends a request by calling execute() on a command object. The command knows how to perform the action on the receiver and can undo it.

Benefits: Decouples sender and receiver, supports undo/redo, queuing, logging, and macro commands.

5. OBSERVER PATTERN

Defines a one-to-many dependency so that when one object changes state, all its dependents are notified and updated automatically.

UML DIAGRAM



EXAMPLE

News Agency



JAVA CODE EXAMPLE

```
// Subject Interface
interface Subject {
  void attach(Observer o);
  void detach(Observer o);
  void notifyObservers();
}

// Observer Interface
interface Observer {
  void update(String news);
}
```

```
// Concrete Subject
class NewsAgency implements Subject {
  private List<Observer> observers =
    new ArrayList<>();
  private String news;

  public void attach(Observer o) {
    obserrers.add(o);
  public void detach(Observer o) {
    obesrrers.remove(o);
  public void setNews(String news) {
    this.news= news;
    notifyObservers();
  }
  public void notifyObservers( {
    for(Observer o : observers{
      o.update(news);
  }
}
```

```
// Concrete Observer
class EmailSubscriber implements Observer {
  private String email;
  public EmailSubscriber(String email) {
    this.email = email;
  public void update(String news) {
    System.out.println("Email to " + email
      + ": " + news);
}
// Another Observer
class SMSSubscriber implements Observer {
  private String phone;
  public SMSSubscriber(String phone) {
    this.phone = phone;
  public void update(String news) {
    System.out.println("SMS to " + phone
      + ": " + news);
}
```

How it works: When the subject's state changes, it notifies all registered observers, and they update themselves.

Benefits: Loosely coupled, supports broadcast communication, easy to add new observers.

What are Behavioral Design Patterns?

Behavioural design patterns focus on **how objects communicate and interact with each other.**

They define how responsibilities and algorithms are distributed among objects.

They help make communication between objects simpler:

- **Flexible**
- **Maintainable**
- **Loosely coupled**
- **Easier to extend**

Key Idea

Behavioural patterns focus on object communication and behavior, rather than object creation or structural organization.

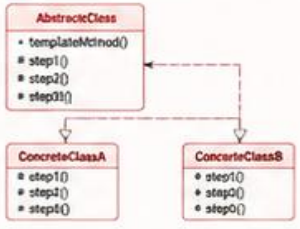
MAJOR BEHAVIOURAL PATTERNS

1. Template Method
2. Chain of Responsibility
3. State Pattern
4. Command Pattern
5. Observer Pattern

1. TEMPLATE METHOD PATTERN

Defines the skeleton of an algorithm in a base class but lets subclasses override specific steps of the algorithm without changing its structure.

UML Diagram



Example



Making tea or coffee follows the same steps (boil water, brew, add condiments) but some steps vary.

Java Code Example

```
abstract class Beverage {
  final void prepareRecipe() {
    boilWater();
    brew();
    pourInCup();
    addCondiments();
  }
  abstract void brew();
  abstract void addCondiments();
  void boilWater() { System.out.println("Boiling water"); }
  void pourInCup() { System.out.println("Pouring into cup"); }
}
class Tea extends Beverage {
  void brew() { System.out.println("Steeping tea"); }
  void addCondiments() { System.out.println("Adding lemon"); }
}
class Coffee extends Beverage {
  void brew() { System.out.println("Dripping coffee"); }
  void addCondiments() { System.out.println("Adding sugar"); }
}
```

Use When

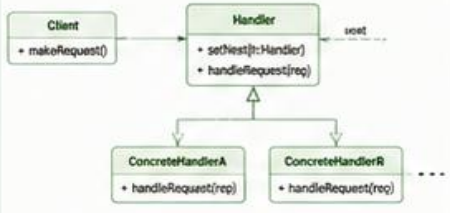


- Multiple classes share the same algorithm structure.
- Some steps of the algorithm can be customized.

2. CHAIN OF RESPONSIBILITY PATTERN

Passes a request along a chain of handlers until one of them handles the request.

UML Diagram



Example



A support request is handled by Support Executive → Team Lead → Manager → Director until someone resolves it.

Java Code Example

```
abstract class Handler {
  protected Handler next;
  public void setNext(Handler next) { this.next = next; }
  public abstract void handleRequest(String req);
}
class SupportExecutive extends Handler {
  public void handleRequest(String req) {
    if(req.equals("Level1"))
      System.out.println("Handled by Support Executive");
    else if(next != null) next.handleRequest(req);
  }
}
class TeamLead extends Handler {
  public void handleRequest(String req) {
    if(req.equals("Level2"))
      System.out.println("Handled by Team Lead");
    else if(next != null) next.handleRequest(req);
  }
}
// Similarly Manager, Director...
```

Use When

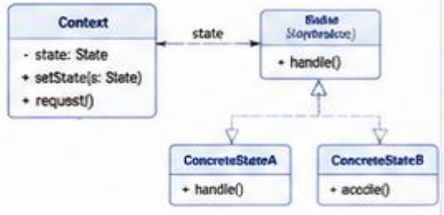


- More than one object can handle a request.
- The handler is not known in advance.
- You want to decouple sender and receiver.

3. STATE PATTERN

Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

UML Diagram



Example



A traffic light changes behavior based on its state: Red → Green → Yellow → Red. Each state has different behavior.

Java Code Example

```
interface State { void handle(TrafficLight t); }
class TrafficLight {
  private State state;
  public void setState(State s) { state = s; }
  public void request() { state.handle(this); }
}

class RedState implements State {
  public void handle(TrafficLight t) {
    System.out.println("Red Light");
    t.setState(new GreenState());
  }
}
// Similarly GreenState, YellowState...
```

Use When

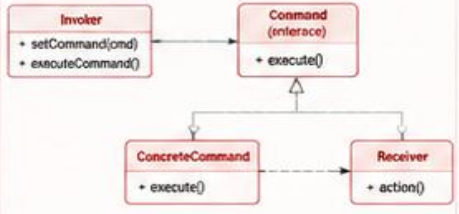


- The behavior depends on the state of an object.
- The code has many conditional statements.
- States should be easy to add or change.

4. COMMAND PATTERN

Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undo.

UML Diagram



Example



A remote control button (Invoker) executes a command (like Turn ON) on a device (Receiver).

Java Code Example

```
interface Command { void execute(); }
class Light {
  public void turnOn() { System.out.println("Light ON"); }
  public void turnOff() { System.out.println("Light OFF"); }
}
class TurnOnCommand implements Command {
  private Light light;
  public TurnOnCommand(Light l) { light = l; }
  public void execute() { light.turnOn(); }
}
class RemoteControl {
  private Command command;
  public void setCommand(Command cmd) { command = cmd; }
  public void pressButton() { command.execute(); }
}
```

Use When

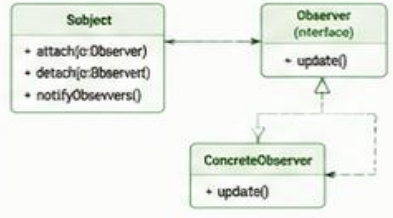


- You want to parameterize objects with requests.
- You want to support undo/redo operations.
- You want to queue or log requests.

5. OBSERVER PATTERN

Defines a one-to-many dependency so that when one object changes state, all its dependents are notified and updated automatically.

UML Diagram



Example



When a user posts on social media, all followers (observers) get notified.

Java Code Example

```
interface Observer { void update(String msg); }
class Subject {
  private List<Observer> observers = new ArrayList<>();
  public void attach(Observer o) { observers.add(o); }
  public void detach(Observer o) { observers.remove(o); }
  public void notifyObservers(String msg) {
    for(Observer o : observers) o.update(msg);
  }
  public void post(String msg) { notifyObservers(msg); }
}
class Follower implements Observer {
  public void update(String msg) {
    System.out.println("New Post: " + msg);
  }
}
```

Use When



- When a change in one object requires automatic updates to others.
- When many objects depend on one object.

BEHAVIOURAL DESIGN PATTERNS (Part–1)

Behavioural patterns focus on how objects communicate and interact with each other. They make communication: **Flexible, Maintainable, Loosely coupled, Easier to extend.**

1. TEMPLATE METHOD PATTERN

Intent:

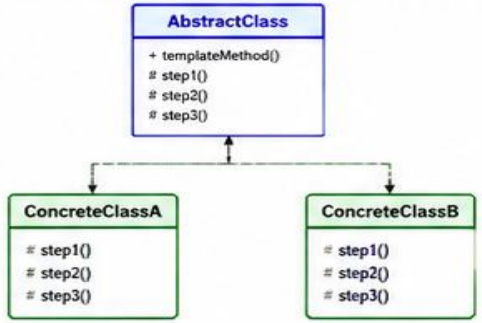
Define the skeleton of an algorithm in a base class but let subclasses override specific steps without changing the algorithm's structure.

When to Use:

- When you want to ensure a consistent algorithm structure.
- When some steps can vary.

Key Benefits:

- ✓ Promotes code reuse.
- ✓ Controls subclass extension.
- ✓ Follows the **Hollywood Principle:** "Don't call us, we'll call you."



Example: Preparing a Beverages

```
// Abstract Class
abstract class Beverage {
    final void prepareRecipe() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }
    void boilWater() { System.out.println("Boiling water"); }
    abstract void brew();
    void pourInCup() { System.out.println("Pouring into cup"); }
    abstract void addCondiments();
}
```

```
class Tea extends Beverage {
    void brew() { System.out.println("Steeping tea"); }
    void addCondiments() { System.out.println("Adding lemon"); }
}
```

```
class Coffee extends Beverage {
    void brew() { System.out.println("Dripping coffee"); }
    void addCondiments() { System.out.println("Adding sugar and milk"); }
}
```

2. CHAIN OF RESPONSIBILITY PATTERN

Intent:

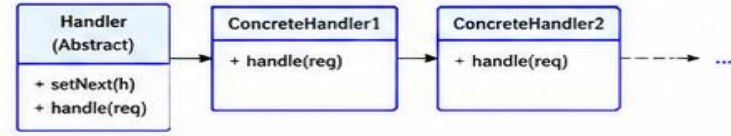
Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request.

When to Use:

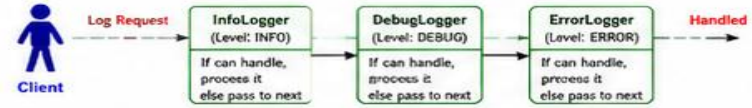
- When multiple objects can handle a request.
- When the set of handlers may change.
- When you want to handle a request without specifying the receiver explicitly.

Key Benefits:

- ✓ Reduces coupling.
- ✓ Flexible in assigning responsibilities.
- ✓ Supports Open/Closed Principle.



Example: Logging System




```
abstract class Logger {
    protected Logger next;
    public void setNext(Logger next) { this.next = next; }
    public abstract void log(String level, String message);
}

class InfoLogger extends Logger {
    public void log(String level, String message) {
        if (level.equals("INFO"))
            System.out.println("INFO: " + message);
        else if (next != null) next.log(level, message);
    }
}
```

```
class DebugLogger extends Logger {
    public void log(String level, String message) {
        if (level.equals("DEBUG"))
            System.out.println("DEBUG: " + message);
        else if (next != null) next.log(level, message);
    }
}

class ErrorLogger extends Logger {
    public void log(String level, String message) {
        if (level.equals("ERROR"))
            System.out.println("ERROR: " + message);
        else if (next != null) next.log(level, message);
    }
}
```

BEHAVIOURAL DESIGN PATTERNS (Part–2)

3. OBSERVER PATTERN

Intent:

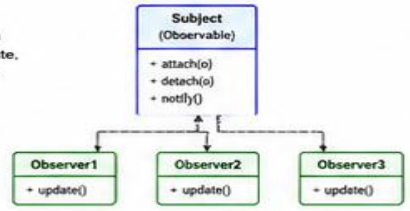
Define a **one-to-many dependency** between objects so that when one object changes state, all its dependents are notified automatically.

When to Use:

- When a change in one object requires changing others.
- When objects should be notified without tight coupling.

Key Benefits:

- ✓ Loosely coupled communication.
- ✓ Easy to add/remove observers.
- ✓ Supports broadcast communication.



Example: News Channel




```
interface Observer {
    void update(String news);
}

class NewsChannel {
    private List<Observer> observers = new ArrayList<>();
    void attach(Observer o) { observers.add(o); }
    void detach(Observer o) { observers.remove(o); }
    void notifyAllObservers(String news) {
        for (Observer o : observers) o.update(news);
    }
}
```

```
class MobileApp implements Observer {
    public void update(String news) {
        System.out.println("Mobile App received: " + news);
    }
}

class EmailSubscriber implements Observer {
    public void update(String news) {
        System.out.println("Email received: " + news);
    }
}
```

4. COMMAND PATTERN

Intent:

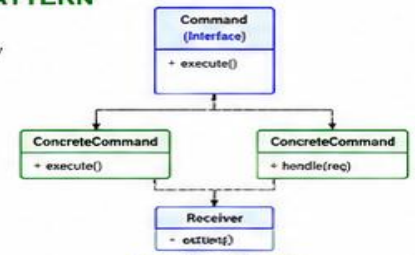
Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

When to Use:

- When you want to parameterize objects with operations.
- When you want to queue, log, or undo operations.

Key Benefits:

- ✓ Decouples sender and receiver.
- ✓ Supports **Undo/Redo.**
- ✓ Easy to extend with new commands.



Example: Remote Control



Example: Remote Control

```
interface Command {
    void execute();
}

class Light {
    void turnOn() { System.out.println("Light ON"); }
    void turnOff() { System.out.println("Light OFF"); }
}
```

```
class LightOnCommand implements Command {
    Light light;
    LightOnCommand(Light l) { light = l; }
    public void execute() { light.turnOn(); }
}

class RemoteControl {
    Command command;
    void setCommand(Command c) { command = c; }
    void pressButton() { command.execute(); }
}
```

5. STATE PATTERN

Intent:

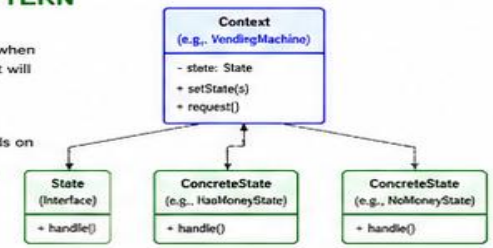
Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

When to Use:

- When an object's behavior depends on its state.
- When you have large conditional statements for state handling.

Key Benefits:

- ✓ Eliminates large switch-case.
- ✓ State transitions are easy to manage.
- ✓ Encapsulates state-specific behavior.






```
interface State {
    void handle(Context context);
}

class HasMoneyState implements State {
    public void handle(Context context) {
        System.out.println("Item selected. Dispensing...");
        context.setState(new DispensingState());
    }
}
```

```
class NoMoneyState implements State {
    public void handle(Context context) {
        System.out.println("Please insert money");
    }
}

class Context {
    private State state;
    void setState(State s) { state = s; }
    void request() { state.handle(this); }
}
```

1. TEMPLATE METHOD PATTERN

Definition: The Template Method Pattern **defines the skeletom of an algorithm in a superclass** while allowing **subclasses to redefine certain steps** of the algorithm without **changing its overall structure.**

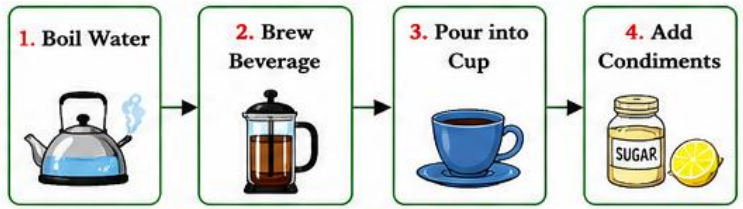
≫ Main Idea

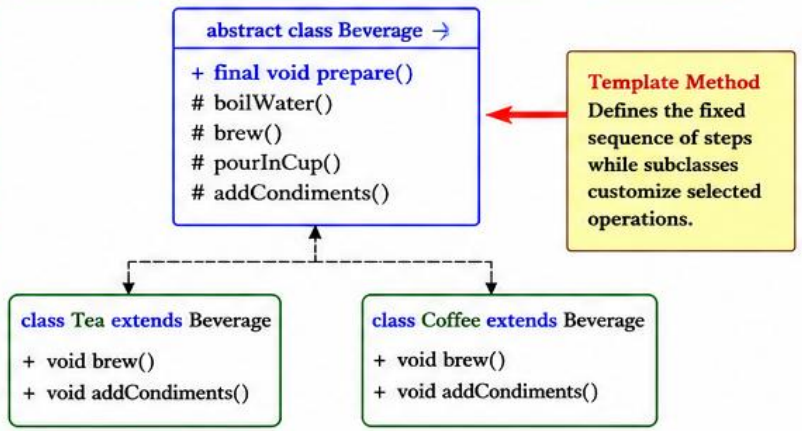
- Algorithm **structure is fixed,** but individual **steps can vary.**

Example: Preparing a Beverage

- A general process may be: Prepare Beverage → Boil Water → Brew Beverage → Pour into Cup → Add Condiments
- **For tea:** Boil Water → Steep Tea → Pour → Add Lemon
- **For coffee:** Boil Water → Brew Coffee → Pour → Add Sugar/Milk
- The overall algorithm remains the **same,** but some steps differ.

Structure: Beverage → Tea / Coffee





Sample Code (Java)

```
abstract class Beverage {
    final void prepare() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }

    abstract void brew();
    abstract void addCondiments();

    void boilWater() {
        System.out.println("Boiling water");
    }
    void pourInCup() {
        System.out.println("Pouring into cup");
    }
}
```

```
class Tea extends Beverage {
    void brew() { System.out.println("Steeping tea"); }
    void addCondiments() { System.out.println("Adding lemon"); }
}
```

```
class Coffee extends Beverage {
    void brew() { System.out.println("Brewing coffee"); }
    void addCondiments() { System.out.println("Adding sugar and milk");}
}
```

Benefits

- ✓ Reuses common algorithm structure.
- ✓ Avoids duplicate code.
- ✓ Allows subclasses to customize specific steps.
- ✓ Maintains a consistent algorithm.
- ✓ Supports code reuse through inheritance.

When to Use

- ≫ Several classes follow the same overall process.
- ≫ Some steps differ between classes.
- ≫ You want to avoid repeating common algorithm code.

How It Works – Execution Example

```
Tea t = new Tea();
t.prepare();
```

Output:

1. Boiling water
2. Steeping tea
3. Pouring into cup
4. Adding lemon



```
Coffee c = new Coffee();
c.prepare();
```

Output:

1. Boiling water
2. Brewing coffee
3. Pouring into cup
4. Adding sugar and milk



In Short:

Template Method Pattern is like a recipe (algorithm) with **fixed steps,** where some steps are **common** (boil water, pour in cup) and some steps are **optional/variable** (brew, add condiments). Subclasses provide their own implementation for the variable steps, while the **overall process remains the same.**

Real-World Analogy

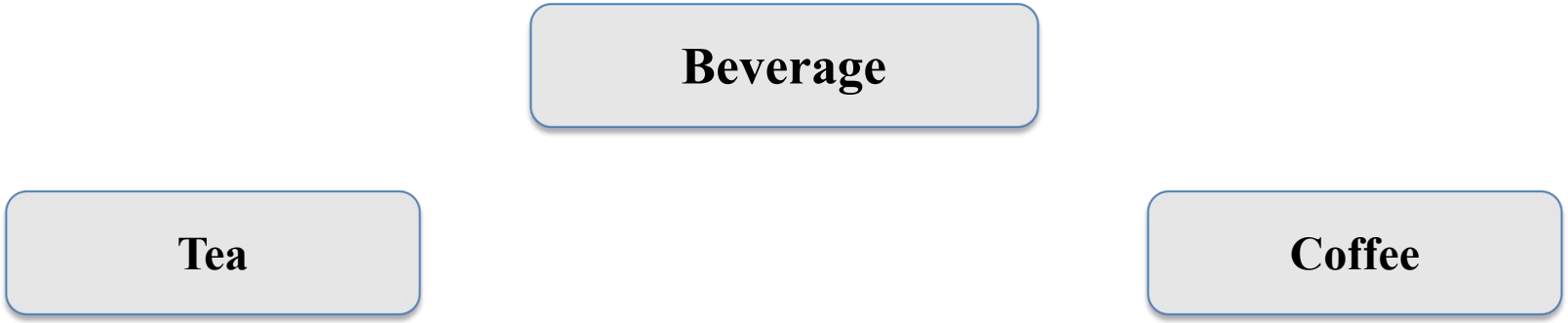
Cooking is the same process, but the ingredients and some steps change depending on the dish.



1. Template Method Pattern — Definition

- **The Template Method Pattern defines the skeleton of an algorithm in a superclass while allowing subclasses to redefine certain steps of the algorithm without changing its overall structure.**
- **Main Idea:** Algorithm structure is fixed, but individual steps can vary.
- **Example:** Preparing a Beverage
- **A general process may be:** Prepare Beverage → Boil Water → Brew Beverage → Pour into Cup → Add Condiments
- **For tea:** Boil Water → Steep Tea → Pour → Add Lemon
- **For coffee:** Boil Water → Brew Coffee → Pour → Add Sugar/Milk
- The overall algorithm remains the same, but some steps differ.

Template Method Pattern — Structure and Benefits



- **Structure:** Beverage → Tea / Coffee
- **Template Method:** Defines the fixed sequence of steps while subclasses customize selected operations.
- **Benefits:** Reuses common algorithm structure; Avoids duplicate code; Allows subclasses to customize specific steps; Maintains a consistent algorithm; Supports code reuse through inheritance.
- **When to Use:** Several classes follow the same overall process; Some steps differ between classes; You want to avoid repeating common algorithm code.

Template Method Pattern — Java Example

```
abstract class Beverage {

    final void prepare() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }

    abstract void brew();
    abstract void addCondiments();

    void boilWater() {
        System.out.println("Boiling water");
    }

    void pourInCup() {
        System.out.println("Pouring into cup");
    }
}
```

2. Chain of Responsibility Pattern — Definition

- ✓**The Chain of Responsibility Pattern passes a request through a chain of handlers until one handler can process the request**.
- ✓**Main Idea:** Give multiple objects a chance to handle a request without coupling the sender to a specific receiver.
- ✓**Example:** University Complaint System: Student → Class Teacher → Department Chairman → Dean → Vice Chancellor
- ✓A complaint is passed along the chain until the appropriate authority handles it.
- ✓**Basic Structure:** Client → Handler A → Handler B → Handler C → Handler D

✓ The Chain of Responsibility Pattern **passes a request through a chain of handlers** until one handler **can process the request.**

Main Idea

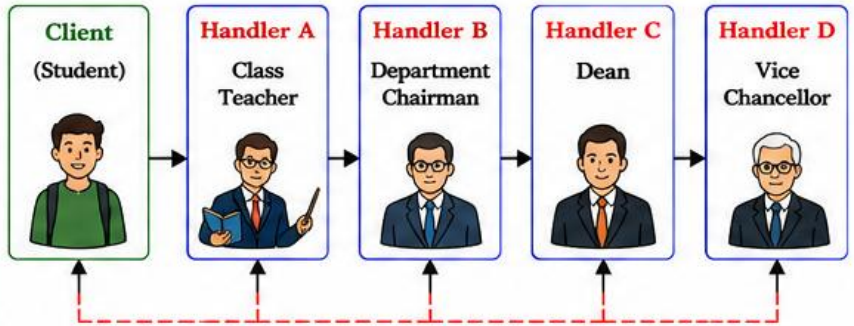
Give multiple objects a chance to handle a request **without coupling** the sender to a specific receiver.

Example: University Complaint System

Student → Class Teacher → Department Chairman → Dean → Vice Chancellor

➢ A complaint is passed along the chain until the **appropriate authority** handles it.

Basic Structure



If cannot handle, pass the request to the next handler.

If handled, the chain stops.

Sample Code (Java)

```
abstract class Handler {
    protected Handler next;
    public void setNext(Handler next) {
       this.next = next;
    }
   public abstract void handleRequest(String request);
}

class ClassTeacher extends Handler {
   public void handleRequest(String request) {
      if (request.equals("academic issue")) {
         System.out.println("Class Teacher handled: " + request);
      } else if (next != null) {
          next.handleRequest(request);
      }
   }
}
// Similarly: DepartmentChairman, Dean, ViceChancellor classes
// Each class decides whether to handle or pass to next handler
```

Benefits

- ✓ Reduces coupling between sender and receiver.
- ✓ Multiple objects can handle a request.
- ✓ Handler order can be changed easily.
- ✓ New handlers can be added without modifying the client.
- ✓ Supports flexible request processing.

How It Works – Step by Step

1. **Student** (client) sends a complaint.
2. Class **Teacher** checks: Can I handle?
   - If **yes** → handles and **stops.**
   - If **no** → passes to next (Department Chairman).
3. **Department Chairman** checks: Can I handle?
   - If **no** → passes to Dean.
4. **Dean** checks: Can I handle?
   - If **no** → passes to Vice Chancellor.
5. **Vice Chancellor** checks: Can I handle?
   - If **yes** → handles and **stops.**
6. If **no** handler can process → request remains **unhandled.**

Example Flow

Complaint: **"hostel facility issue"**

↓

Stodent

↓

Class Teacher (**cannot handle**)

↓

Department Chairman (**cannot handle**)

↓

Dean (**cannot handle**)

↓

Vice Chancellor (handles)

✅ **Request Handled**

Common Applications

- ✓ Approval systems
- ✓ Exception handling
- ✓ Customer support systems
- ✓ Authentication / authorization
- ✓ Event processing

In Short

Chain of Responsibility Pattern forms a "**chain**" of objects.

A request **travels** along the chain until an object handles it.

If no one handles it, the request **remains unhandled.**

Real-Life Analogy



When you have a problem in a **bank**,
you first talk to the **staff**.
If they cannot solve it,
they forward you to the **manager**,
then to the **branch head**,
and finally to the **regional manager** if needed.

Key Points to Remember

- ✓ Request passes along a **chain**.
- ✓ Each handler decides: **handle** or pass.
- ✓ Loose coupling between client and handlers.
- ✓ Easy to add, remove, or reorder handlers.
- ✓ More flexible and maintainable code.

Chain of Responsibility — Java Structure and Benefits

Benefits:

- ✓Reduces coupling between sender and receiver;
- ✓Multiple objects can handle a request;
- ✓Handler order can be changed easily;
- ✓New handlers can be added without modifying the client;
- ✓Supports flexible request processing.
- ✓Common Applications: Approval systems;
- ✓ Exception handling; Customer support systems;
- ✓Authentication/authorization; Event processing.

3. State Pattern — Definition and Example

- ✓ **The State Pattern allows an object to change its behavior when its internal state changes.**
- ✓ **Main Idea:** The object appears to change its class when its state changes.
- ✓ **Example:** Online Order
- ✓ An order can have different states: Pending; Paid; Shipped; Delivered.
- ✓ The behavior of the order depends on its current state.



STATE PATTERN

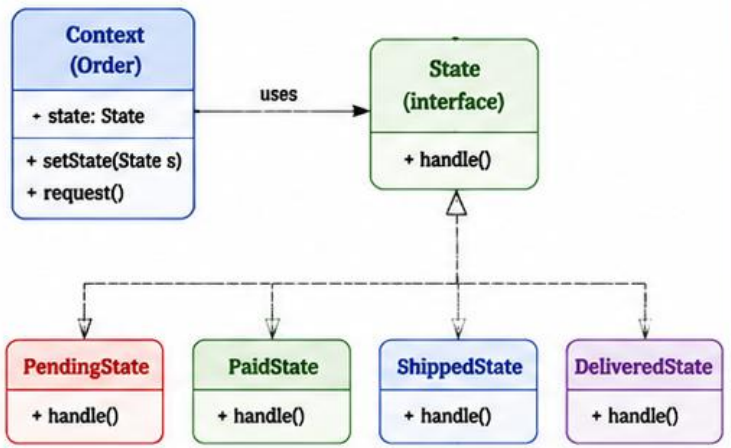
✔ The State Pattern **allows an object to change its behavior when its internal state changes.**

✔ **Main Idea:** The object appears to change its class when its state changes.

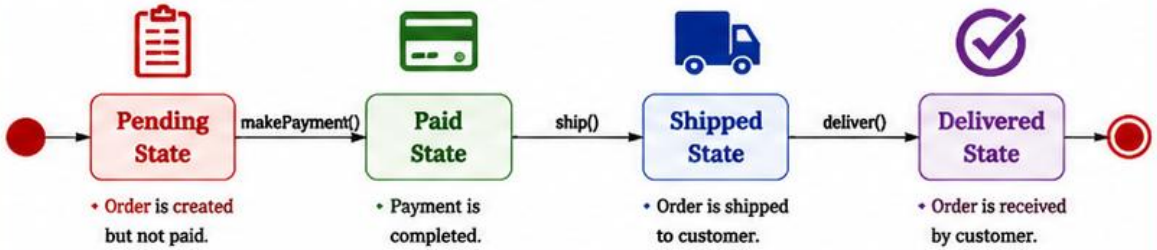
EXAMPLE: ONLINE ORDER

- An order can have different states: **Pending**; **Paid**; **Shipped**; **Delivered**.
- The behavior of the order depends on its current state.

STRUCTURE



STATE TRANSITION DIAGRAM (ONLINE ORDER)



JAVA CODE EXAMPLE

```
interface State {
    void handle();
}
class PendingState implements State {
    public void handle() {
        System.out.println("Order is pending");
    }
}
class PaidState implements State {
    public void handle() {
        System.out.println("Order has been paid");
    }
```

```
class Order {
    private State state;
    public Order() {
        state = new PendingState();
    }
    public void setState(State state) {
        this.state = state;
    }
    public void request() {
        state.handle();
    }
}
```

```
// Client
public class Client {
    public static void main(String[] args) {
        Order order = new Order();
        order.request();  // Order is pending
        order.setState(new PaidState());
        order.request();  // Order has been paid
        order.setState(new ShippedState());
        order.request();  // Order is shipped
        order.setState(new DeliveredState());
        order.request();  // Order is delivered
    }
}
```

BENEFITS

- **Encapsulates state-specific behavior.**
- **Reduces large if-else or switch statements.**
- **Makes state transitions easier to manage.**
- **New states can be added easily.**
- **Improves maintainability.**

WHEN TO USE

Use State Pattern when an object's behavior changes significantly depending on its current state.

HOW IT WORKS

1. The **Context** (Order) holds a reference to a **State** object.
2. The current **State** object handles the **request()** based on its behavior.
3. When a state transition occurs, the **Context** changes its **State** object.
4. The new **State** object handles future requests accordingly.

State Pattern — Structure and Benefits



- **Structure:** Context → Current State → Pending / Paid / Shipped
- **Benefits:** Encapsulates state-specific behavior; Reduces large if-else or switch statements; Makes state transitions easier to manage; New states can be added easily; Improves maintainability.
- **When to Use:** Use State Pattern when an object's behavior changes significantly depending on its current state.

State Pattern — Java Example

```
interface State {
    void handle();
}

class PendingState implements State {
    public void handle() {
        System.out.println("Order is pending");
    }
}

class PaidState implements State {
    public void handle() {
        System.out.println("Order has been paid");
    }
}
```

4. Command Pattern — Definition

The Command Pattern encapsulates a request as an object.

This allows requests to be: Stored; Scheduled; Queued; Executed; Undone; Redone; Logged.

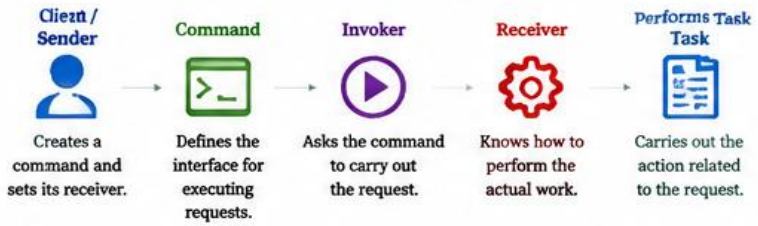
Main Components: Client/Sender → Command → Invoker → Receiver → Performs Task

Example: Text Editor: User → Paste Button → PasteCommand → Document

COMMAND PATTERN

Definition: The Command Pattern encapsulates a request as an object.
This allows requests to be: **Stored**; **Scheduled**; **Queued**; **Executed**; **Undone**; **Redone**; **Logged**.

1. MAIN COMPONENTS



Example: Text Editor



2. MAJOR PURPOSES

1 Store and Schedule Requests

Commands can be stored in lists or queues and executed later.



2 Undo and Redo

Commands can maintain a history so that previous operations can be undone and redone.



3 Logging

Commands can be stored in a log so recent operations can potentially be replayed after a system failure.

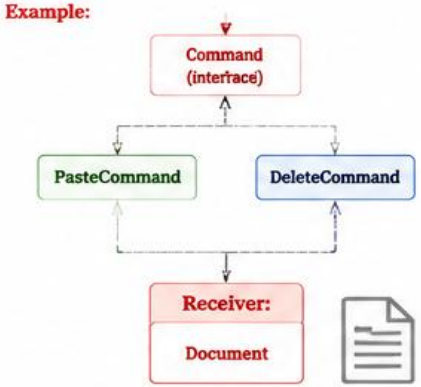


3. COMMAND INTERFACE / SUPERCLASS

```
interface Command {
  void execute();
  void unexecute();
  boolean isReversible();
}
```

- **execute()** → performs the command
- **unexecute()** → reverses the command
- **isReversible()** → determines whether the command can be undone

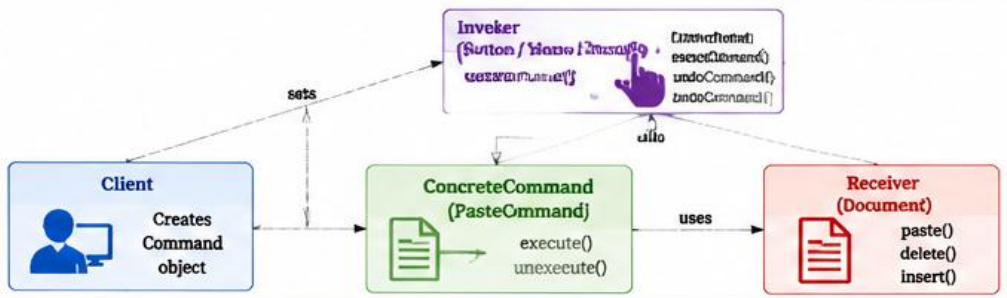
Example:




```
class PasteCommand extends Command {
  privete Document doc;
  privete String content;
  public void execute() { doc.paste(content); }
  public void unexecute() { doc.delete(content); }
  public boolean isReversible() { return true; }
}
class DeleteCommand extends Command {
  privete Document doc;
  privete int index; privete String backup;
  public void execute() { backup = doc.delete(index); }
  public void unexecute() { doc.insert(index, backup); }
  public boolean isReversible() { return true; }
}
```

4. HOW IT WORKS (FLOW)

1. Client creates a concrete command object and sets the receiver.
2. Invoker stores the command object.
3. Invoker calls execute() on the command.
4. Command calls the receiver to perform the actual task.
5. (Optional) Command can be unexecute() to undo the task.



5. BENEFITS

1. **Commands become objects** — Requests can be manipulated as objects.
2. **Supports Undo/Redo** — Commands can maintain history and reversal operations.
3. **Supports Queueing** — Commands can be placed in queues and executed later.
4. **Reduces Coupling** — The command object separates the requester from the receiver.
5. **Separates UI from Application Logic** — Button clicks can create command objects instead of directly containing application logic.
6. **Easy to Maintain** — New commands can be added independently.

Command Pattern — Purpose

Major Purposes

1. Store and Schedule Requests

Commands can be stored in lists or queues and executed later.

Example: Create Alarm Command → Command Queue → Scheduled Time → Execute Alarm

2. Undo and Redo

Execute → History List → Undo → Redo List

Commands can maintain a history so that previous operations can be undone and redone.

3. Logging

Commands can be stored in a log so recent operations can potentially be replayed after a system failure.



Command Pattern — Structure

Command Interface / Superclass

- execute() → performs the command
- unexecute() → reverses the command
- isReversible() → determines whether the command can be undone

Example: Command → PasteCommand / DeleteCommand → Receiver: Document

Command Pattern — Benefits

1. **Commands become objects** — Requests can be manipulated as objects.
2. **Supports Undo/Redo** — Commands can maintain history and reversal operations.
3. **Supports Queueing** — Commands can be placed in queues and executed later.
4. **Reduces Coupling** — The command object separates the requester from the receiver.
5. **Separates UI from Application Logic** — Button clicks can create command objects instead of directly containing application logic.
6. **Easy to Maintain** — New commands can be added independently.



5. Observer Pattern — Definition and Example

The Observer Pattern defines a one-to-many relationship in which a Subject maintains a list of Observers and automatically notifies them when its state changes.

Main Idea: One object changes → many dependent objects are notified.

Example: Blog Subscription: Blog (Subject) → Subscriber 1 / Subscriber 2 / Subscriber 3 (Observers)

When the blog changes: Blog Updated → notify() → update() on all subscribers.

```
interface Observer { void update(); }

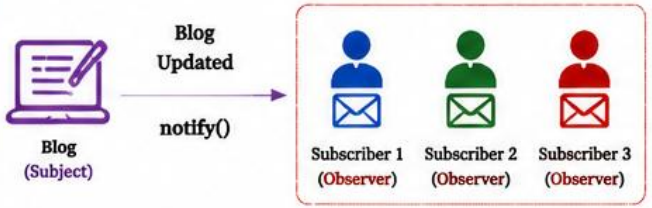
Subject: registerObserver(); unregisterObserver(); notify();
```

OBSERVER PATTERN

Definition: The Observer Pattern defines a one-to-many relationship in which a Subject maintains a list of Observers and automatically notifies them when its state changes.
Main Idea: One object changes → many dependent objects are notified.

1. EXAMPLE: BLOG SUBSCRIPTION

Blog (Subject) → Subscriber 1 / Subscriber 2 / Subscriber 3 (Observers)



2. MAIN ROLES AND OPERATIONS

Subject
Maintains a collection of observers.
Main operations:
- registerObserver(Observer o)
- unregisterObserver(Observer o)
- notify()

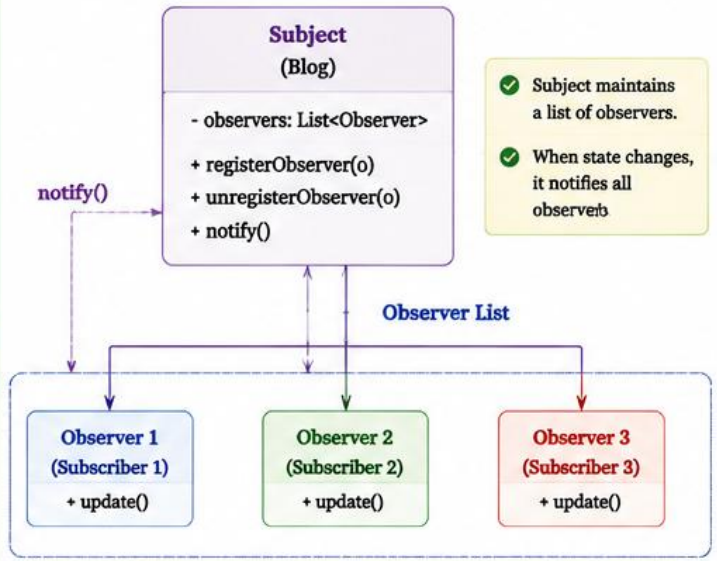
Observer
Defines update().

3. INTERFACE (EXAMPLE)

```
interface Observer {
    void update();
}

interface Subject {
    void registerObserver(Observer o);
    void unregisterObserver(Observer o);
    void notify();
}
```

4. STRUCTURE DIAGRAM



notify() calls update() on all registered observers

5. DYNAMIC SUBSCRIBE / UNSUBSCRIBE



Observers can subscribe and unsubscribe dynamically.
The Subject does not need to know the concrete classes of observers.

6. JAVA CODE EXAMPLE

```
import java.util.*;

interface Observer {
    void update();
}

class Subject {
    private ArrayList<Observer> observers =
            new ArrayList<>();

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void unregisterObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers) {
            o.update();
        }
    }
}
```

7. KEY BENEFITS

- Establishes a one-to-many dependency between objects.
- Loose coupling between Subject and Observers.
- Automatic notification when state changes.
- Supports dynamic addition and removal of observers.
- Easy to extend: new observers can be added without changing the subject.
- Widely used in event handling, UI frameworks, messaging systems, etc.

Use Observer Pattern when a change in one object requires changing many others, and the number of dependent objects is unknown or changes at runtime.

Real World Examples: GUI event handling, stock price updates, weather stations, news notifications, social media feeds, etc.

Key Takeaway: One changes, many get notified!
Keep your system flexible, scalable and decoupled.

Observer Pattern — Structure

Subject: Maintains a collection of observers.

Main operations: registerObserver(); unregisterObserver(); notify().

Observer: Defines update().

Structure: Subject → Observer List → notify() → Observer 1, Observer 2, Observer 3 → update()

Observers can subscribe and unsubscribe dynamically.

Observer Pattern — Java Example

```
class Subject {
    private ArrayList<Observer> observers =
        new ArrayList<>();

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void unregisterObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers) {
            o.update();
        }
    }
}
```

Observer Pattern — Benefits & Applications

Benefits: Automatically notifies dependent objects; Reduces direct dependency between subject and observers; Supports one-to-many communication; Observers can subscribe/unsubscribe dynamically; Makes notification management easier; Useful when many objects depend on one object's state.

Applications: Blog/news subscriptions; GUI event handling; Notification systems; Stock-price monitoring; Weather monitoring; Real-time dashboards.

Comparison of Behavioural Patterns

Template Method → How an algorithm is performed (Same algorithm, variable steps)

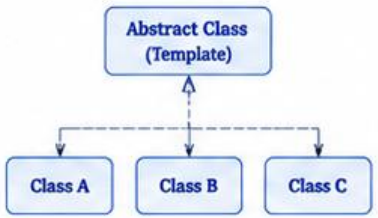
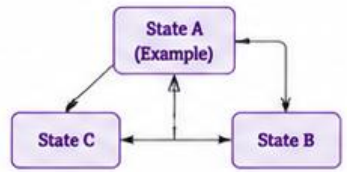
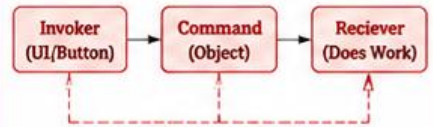
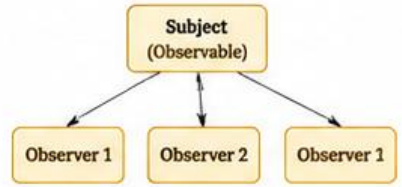
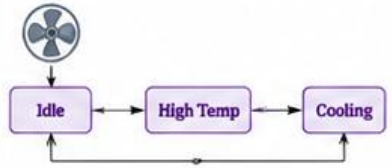
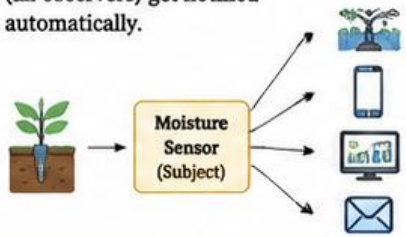
Chain of Responsibility → Who will handle the request? (Handler chain)

State → What should I do in this state? (State-dependent behavior)

Command → Turn a request into an object (Request → Object)

Observer → Who needs to know about a change? (One-to-many notification)

COMPARISON OF BEHAVIORAL DESIGN PATTERNS

	1. TEMPLATE METHOD PATTERN	2. CHAIN OF RESPONSIBILITY PATTERN	3. STATE PATTERN	4. COMMAND PATTERN	5. OBSERVER PATTERN
What it is (All about)	**How** an algorithm is performed (Same algorithm, variable steps)	**Who** will handle the request? (Handler chain)	**What** should I do in this state? (State-dependent behavior)	**Turn** a request into an object (Request → Object)	**Who** needs to know about a change? (One-to-many notification)
Core Idea	Define the skeleton of an algorithm in a base class and let subcleases implement specific steps.	Pass a request along a chain of handlers until one handler processes it.	Allow an object to change its behavior when its internal state changes.	Encapsulate a request as an object so it can be stored, queued, executed, undone, or logged.	Define a one-to-many dependency so that when one object changes state, all its dependents are notified.
Structure (Visual)		 Request comes → Passed along until handled	 Object changes state → behavior changes	 Execute / Undo / Redo / Log	 Change in Subject → notify() → update() all observers
Real-life Example (Agricultural Automation System)	**Irrigation Scheduling System** The system has a fixed algorithm: Check soil moisture → Check weather forecast → Decide irrigation → Log result. Different crop types (Rice, Wheat, Maize) inplement how they calculate "Decide irrigation" step differently. 	**Alarm Handling System** When an alarm occurs (e.g., high temperature), it goes through a chain: Sensor Check → Local Controller → Farm Manager → Cloud Server. Whoever can handle the alarm, handles it. 	**Greenhouse Ventilation System** The fan behavior depends on the state: Idle → Low Temp → High Temp → Cooling. The system changes its state based on temperature and acts accordingly. 	**Farm Operation Control System** User presses "Start Pump" button. A StartPumpCommand object is created and sent to the pump controller. The command can be stored, queued, executed later, undone (stop pump) or logged.  Undo / Redo / Log	**Soil Moisture Monitoring System** When soil moisture value changes, many components should know: Irrigation Controller, Mobile App, Dashboard, and SMS Alert Service (all observers) get notified automatically. 
Best When to Use	When the overall algorithm is fixed but certain steps vary between implementations.	When multiple objects can handle a request and you don't know in advance who will handle it.	When an object's behavior changes based on its internal state.	When you need to parameterize objects with operations, support undo/redo, queue, or logging.	When a change in one object needs to be broadcast to many dependent objects.

DHAKA
Urban Management

Behavioral Design Patterns in a Dhaka Urban Management System Project

One Smart City – Five Behavioral Pattens – Five Real Solutions

Pattern	1. Template Method Pattern	2. Chain of Responsibility Pattern	3. State Pattern	4. Command Pattern	5. Observer Pattern
Key Idea	How an algorithm is performed (Same algorithm, variable steps)	Who will handle the request? (Handler chain)	What should I do in this state? (State-dependent behavior)	Turn a request into an object (Request → Object)	Who needs to know about a change? (One-to-many notification)
Dhaka Project Scenario	**Daily Waste Collection** Different zones of Dhaka follow the same waste collection process, but each zone may implement specific steps differently.	**Citizen Complaint Handling** A complaint is passed through departments until the right authority handles it.	**Bus / Metro Ticket Status** A ticket system behaves differently depending on its current state.	**Traffic Signal Control** Each operation (change signal, turn on camera, open barrier) is a command that can be queued, scheduled, logged and undone.	**Air Quality Monitoring** When air quality changes, all relevant departments and citizens are automatically notified.
How it Works		 Request passes through a chain until the responsible office handles			
Main Participants	Abstract Template (Base Process) → Zone A / Zone B / Zone C	ComplaintHandler → CallCenter → WardOffice → CityCorporation → SpecialAuthority	Ticket (Context) → State (Issued / Used / Expired / Blocked)	Invoker (Traffic App) → Command → Receiver (Traffic Controller)	Subject → Observer List → notify() → All Observers

Code (Simplified)	`abstract class WasteCollection {`
	`  pubic final void collect() {`
	`  planRoute();`
	`  collectWaste(); // variable step`
	`  transport();`
	`  dispose();`
	`  }`
	`  protected abstract void collectWaste();`
	`}`
		`interface ComplaintHandler {`
		`  void setNext(ComplaintHandler h);`
		`  void handle(Complaint c);`
		`}`
			`interface TicketState {`
			`  void process(Card c);`
			`}`
			`class IssuedState implements`
			`  TicketState { ... }`
				`interface Command {`
				`  void execute();`
				`  void unexecute(); }`
				`}`
				`class StartGreenCommand`
				`  implements Command { ... }`
					`class AirQualitySubject {`
					`  private List<Observer> obs =`
					`  new ArrayList<<();`
					`void registerObserver(Observer o) {...}`
					`void notifyObservers() {`
					`  for (Observer o : obs) o.update();`
					`}`

Benefit	Ensures a standard process yet allows zone-specific customization.	Flexible and scalable complaint processing.	Keeps system behavior clear and easy to extend for new states (e.g., refunded).	Supports scheduling, logging and undo/redo of traffic control actions.	Ensures real-time information sharing with all stakeholders automatically.

Smart Dhaka, Smarter Management – Design Patterns Make Urban Systems Flexible, Scalable and Maintainable.

Problem Scenario

Suppose You are designing a system for money withdrawal from bank account using check with the following authorization scenario:

- ✓ For Tk. < 10,000 only cashier's authorization is sufficient to withdraw money from account.
- ✓ For Tk. 10,000 to 10,00,000 authorization from both cashier and senior officer is needed.
- ✓ For Tk > 10,00,000 authorization from Senior officer and Manager is needed.
- ✓ **Implement this behaviour using appropriate software design pattern.**

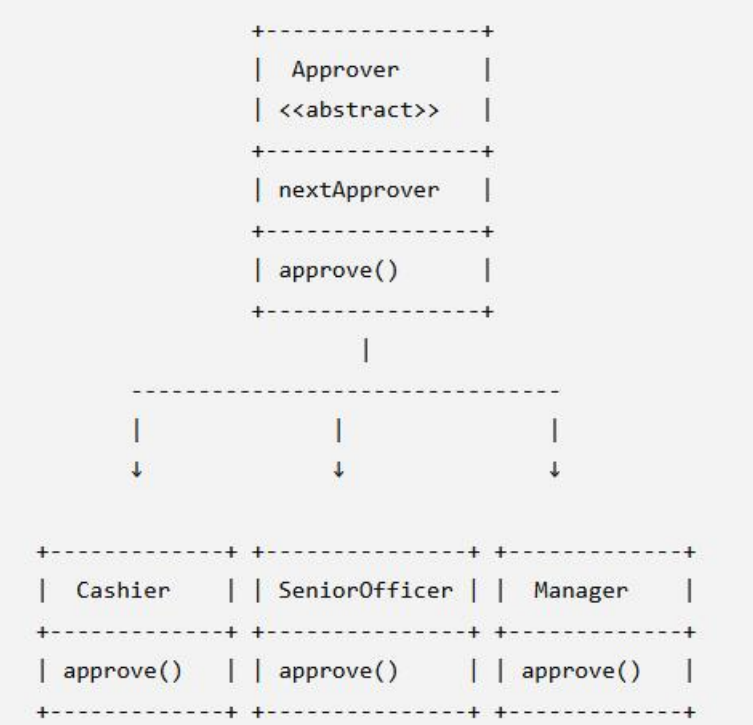
This Scenario is a classic example of the **Chain of Responsibility Design Pattern**.
The reason is:

- ✓ A withdrawal request passes through a sequence of authorization levels.
- ✓ Each authorization object decides whether it can handle the request.
- ✓ If it cannot handle it, it forwards the request to the next authority.
- ✓ The sender does not need to know which object will finally approve the request.

1. Concept

The **Chain of Responsibility pattern** creates a chain of handler objects. Each handler checks whether it can process a request; otherwise, it passes the request to the next handler in the chain.
In this bank withdrawal system:
Cashier | ↓ Senior Officer | ↓ Manager

UML



Abstract Handler: abstract class Approver {
protected Approver nextApprover;
public void setNextApprover(Approver approver)
{
this.nextApprover = approver; }
public abstract void approve(double amount);}

Cashier Class: class Cashier extends Approver{
public void approve(double amount){
if(amount < 10000){
System.out.println("Cashier approved withdrawal of Tk. " + amount); }
else if(nextApprover != null){
nextApprover.approve(amount); } }}

class SeniorOfficer extends Approver {
public void approve(double amount){
if(amount >= 10000 && amount <= 1000000){
System.out.println("Senior Officer approved withdrawal of Tk. " + amount); }
else if(nextApprover != null)
{ nextApprover.approve(amount); } }}

UML and Java Implementation

```
class Manager extends Approver {
 public void approve(double amount) {
 if(amount > 1000000){
        System.out.println(        "Manager approved
withdrawal of Tk. "        + amount);      }
      else {
        System.out.println(        "Withdrawal cannot be
approved");      }   }}
```

Output: Cashier approved withdrawal of Tk. 5000Senior Officer approved withdrawal of Tk. 50000Manager approved withdrawal of Tk. 5000000

```
public class BankSystem {
public static void main(String args[]) {
 Approver cashier = new Cashier();      Approver
senior = new SeniorOfficer();      Approver
manager = new Manager();      // Create chain
cashier.setNextApprover(senior);
senior.setNextApprover(manager);      //
Withdrawal requests      cashier.approve(5000);
cashier.approve(50000);
cashier.approve(5000000);   }}
```

Real World Component	Design Pattern Component
Cashier	First Handler
Senior Officer	Second Handler
Manager	Final Handler
Withdrawal Request	Request Object
Authorization Process	Chain of Responsibility

UML and Java Implementation

Why Chain of Responsibility is Appropriate?

1.Flexible authorization flow

If the bank introduces another authority, for example:
Cashier | Senior Officer | Assistant Manager | Manager
we can add a new handler without modifying existing classes.

2.Loose coupling

The withdrawal request does not directly depend on Cashier, Senior Officer, or Manager.

3.Follows Open/Closed Principle

New approval levels can be added without changing existing code.

Problem -2 Scenario

Question 1: Dhaka Urban Management System (DUMS)
Scenario: Smart Citizen Complaint Notification System
The Dhaka Urban Management System receives thousands of citizen complaints every day regarding waste collection, traffic problems, water supply, and emergency services.
When the status of a complaint changes, different users should automatically receive updates:

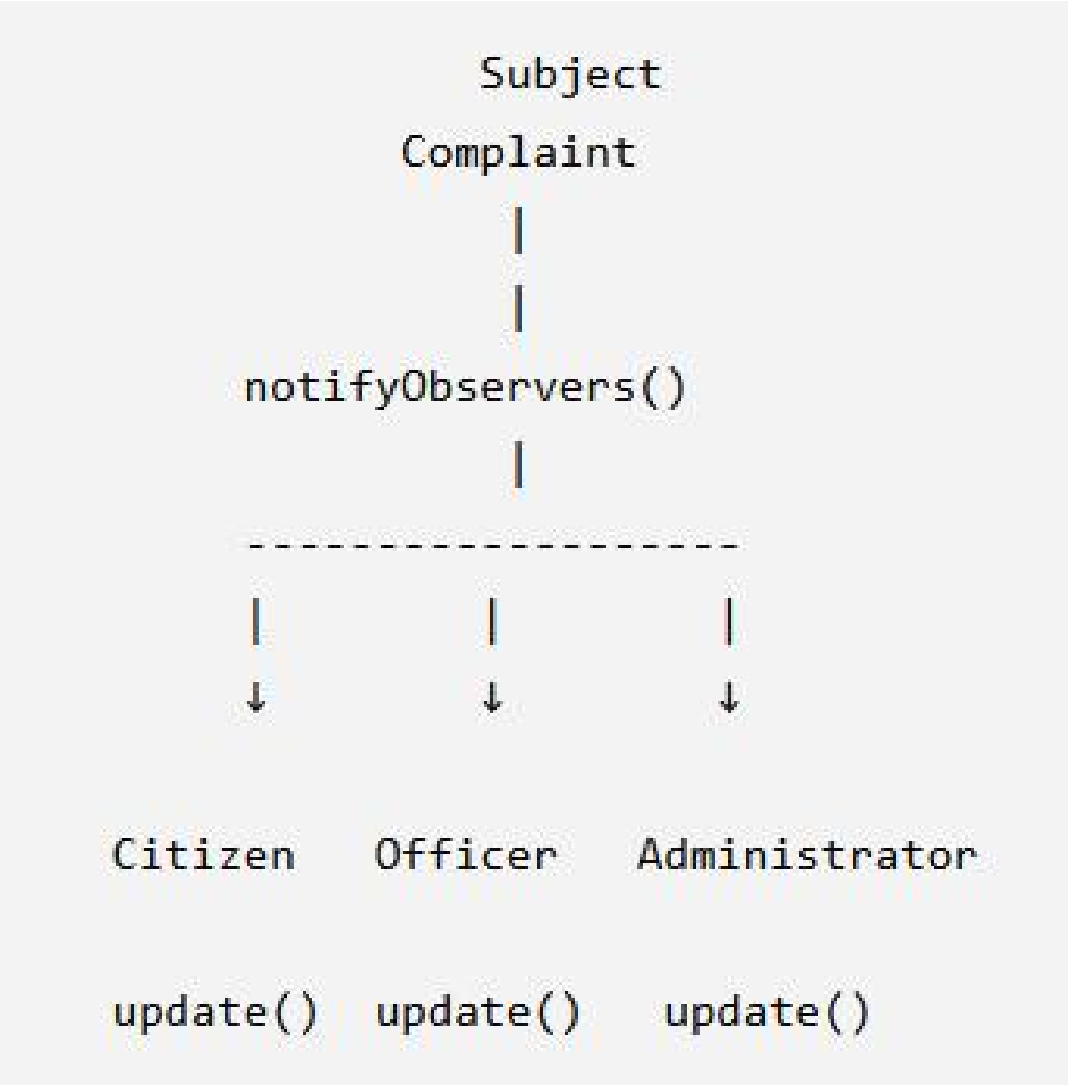
- Citizen receives complaint status notification.
- Department officer receives update information.
- City administrator receives monitoring alerts.

The complaint management system should notify all registered users automatically whenever the complaint status changes.
Requirement:
Implement this behavior using an appropriate software design pattern.

Design Pattern: Observer Pattern
Reason:
The Observer Pattern is suitable because:

- One object (**Complaint**) changes its state.
- Multiple dependent objects (**Citizen, Officer, Administrator**) need automatic updates.
- The complaint system should not tightly depend on specific notification users.

Problem -2 : UML and Java Implementation



Observer Interface

```
interface Observer {
  void update(String message);}
```

Concrete Observer:

```
class Citizen implements Observer {
 public void update(String message){
  System.out.println(       "Citizen Notification: " + message);
 }}
class Officer implements Observer {    public void update(String
message){       System.out.println(       "Officer Alert: " + message);    }}
class Administrator implements Observer {    public void
update(String message){       System.out.println(       "Admin
Dashboard Update: " + message);    }}
```

Problem -2 : UML and Java Implementation

Subject Class

```
import java.util.*;
class Complaint {
ArrayList<Observer> observers =new ArrayList<>();
public void addObserver(Observer o){
 observers.add(o);}
public void changeStatus(String status){    for(Observer o: observers){
o.update(       "Complaint status changed to "       + status);    }}}
```

Main Class

```
public class DUMS {
public static void main(String args[]){
Complaint complaint=new Complaint();
complaint.addObserver(new Citizen());
complaint.addObserver(new Officer());
complaint.addObserver(new Administrator());
complaint.changeStatus("Completed");}}
```

Citizen Notification:

Complaint status changed to CompletedOfficer Alert:
Complaint status changed to CompletedAdmin Dashboard Update:
Complaint status changed to Completed

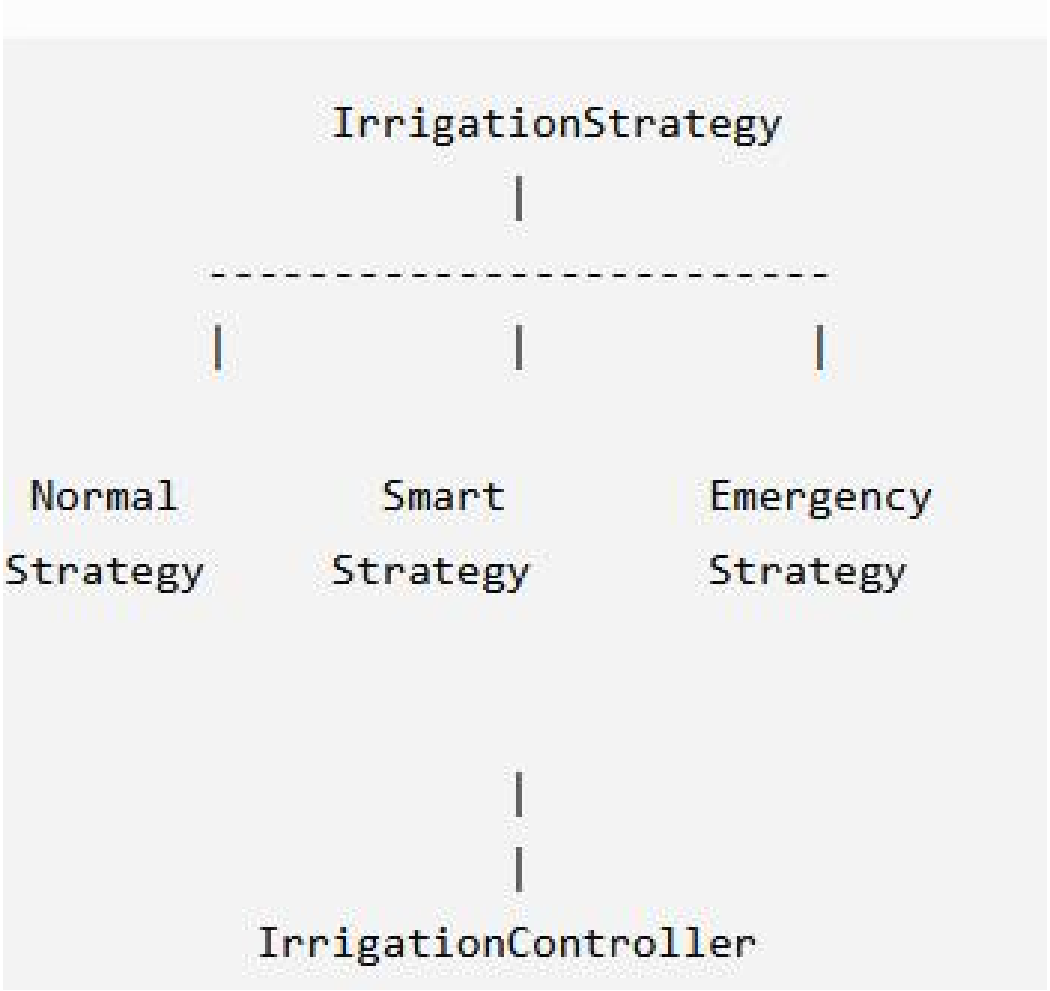
Problem -3 Scenario

Question 2: Agricultural Automation System (AAS)
Scenario: Intelligent Irrigation Strategy Selection
An Agricultural Automation System controls irrigation based on different farming conditions.
The farmer can select different irrigation strategies:
1.Normal irrigation
2.Water-saving irrigation
3.Emergency irrigation during drought
The system should allow changing irrigation algorithms without modifying the main irrigation controller.
Requirement:
Implement this behavior using an appropriate software design pattern.

Solution
Design Pattern: Strategy Pattern
Reason:
Strategy Pattern is suitable because:

- Different irrigation algorithms exist.
- The algorithm can change dynamically.
- The irrigation controller does not need to know the internal implementation.

UML and Java Implementation for Scenario-3



interface IrrigationStrategy
{void irrigate();}

class NormalIrrigation implements IrrigationStrategy{
public void irrigate(){
System.out.println("Normal irrigation activated");}}

class WaterSavingIrrigationimplements IrrigationStrategy{public void irrigate(){System.out.println("Water saving irrigation activated");}}

class EmergencyIrrigationimplements IrrigationStrategy{public void irrigate(){System.out.println("Emergency irrigation activated");}}

Problem - 3: UML and Java Implementation

Context class:

```
class IrrigationController{
private IrrigationStrategy strategy;
public void setStrategy(IrrigationStrategy
strategy){this.strategy=strategy;}public void execute(){
strategy.irrigate();}}
```

Citizen Notification:

Water saving irrigation activatedEmergency irrigation activated

Main Class

```
public class FarmSystem{public static void main(String
args[]){IrrigationController controller =new
IrrigationController();controller.setStrategy(new
WaterSavingIrrigation());controller.execute();controller.setStrategy(new
EmergencyIrrigation());controller.execute();}}
```

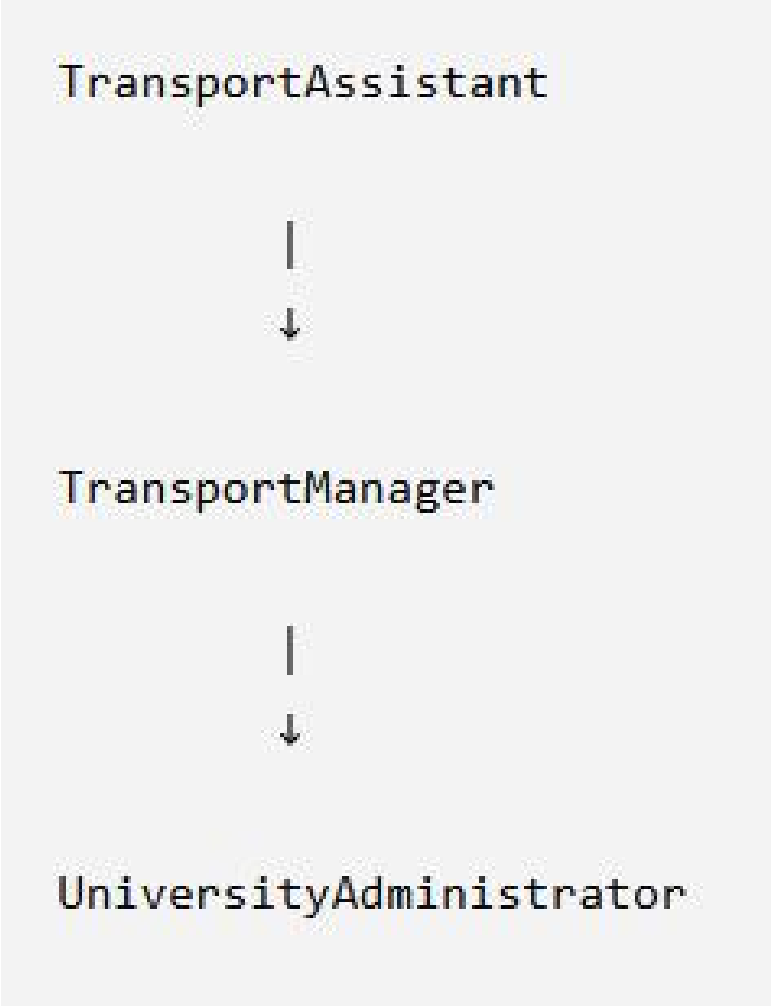
Problem -4 Scenario

Scenario: University Transport Approval System
A smart campus transportation system manages bus ticket approval.
Different transport requests require different approval levels:
•Student transport request below 500 BDT → Transport Assistant approval
•Request from 500 to 5000 BDT → Transport Manager approval
•Request above 5000 BDT → University Administrator approval
The request should automatically move through the approval chain until the responsible authority handles it.
Requirement:
Implement this behavior using an appropriate software design pattern.

Solution
Design Pattern: Chain of Responsibility Pattern

Reason:
The Chain of Responsibility pattern is suitable because:
•Multiple approval levels exist.
•Each handler decides whether it can process the request.
•The request moves to the next authority if required.

UML and Java Implementation for Scenario-4



abstract class Approver{protected Approver next;public void setNext(Approver next){this.next=next;}public abstract void approve(double amount);}

Transport Assistant:

```
class Assistant extends Approver{public void
approve(double
amount){if(amount<500){System.out.println("
Assistant approved
request");}else{next.approve(amount);}}}
```

Transport Manager:

```
class Manager extends Approver{public void
approve(double
amount){if(amount<=5000){System.out.printl
n("Manager approved
request");}else{next.approve(amount);}}}
```

UML and Java Implementation for Scenario-4

Adminstrator:

```
class Administrator extends Approver{public
void approve(double
amount){System.out.println("Administrator
approved request");}}
```

Main:

```
public class CampusTransport{public static
void main(String args[]){Approver
assistant=new Assistant();Approver
manager=new Manager();Approver
admin=new
Administrator();assistant.setNext(manager);ma
nager.setNext(admin);assistant.approve(700
0);}}
```

Output:

Administrator approved request

System	Scenario	Behavioral Pattern
Dhaka Urban Management System	Complaint status notification	Observer Pattern
Agricultural Automation System	Irrigation algorithm selection	Strategy Pattern
Smart Campus Transportation System	Transport approval workflow	Chain of Responsibility Pattern

These three questions together cover three major behavioral patterns:

1.Observer → Object communication

2.Strategy → Algorithm selection

3.Chain of Responsibility → Request processing workflow

They are appropriate for an Object-Oriented Design Pattern Lab exam because students can identify the pattern from the real-world scenario and implement it using Java.

Other Pattern Scenario

Observer Patttern Scenario:
In the Dhaka Urban Management System, citizens can submit complaints regarding waste collection, traffic problems, water supply, and emergency services.
Whenever the complaint status changes (e.g., Submitted → Processing → Completed), multiple users should automatically receive updates:
•Citizen receives notification about complaint progress.
•Responsible department officer receives task updates.
•City administrator receives monitoring information.
The complaint management system should notify all registered users automatically without modifying the complaint class whenever a new type of user is added.
Requirement:
What problem does the Observer Design Pattern solve? Explain how the Observer Pattern can be applied in the Dhaka Urban Management System. Draw the supporting UML diagram and implement the solution.
Expected Pattern: Observer Pattern
Concept:
One object (Subject) maintains a list of dependent objects (Observers) and automatically notifies them when its state changes.

Other Pattern Scenario

Strategy Pattern
Agricultural Automation System (AAS)
Scenario:An Agricultural Automation System controls irrigation based on different farming conditions.The irrigation controller supports different irrigation strategies:Normal irrigation during regular weather conditions.Water-saving irrigation during water shortage.Emergency irrigation during drought situations.The system should allow farmers to change the irrigation strategy dynamically without modifying the irrigation controller code.Requirement:What problem does the Strategy Design Pattern solve? Explain how the Strategy Pattern can be used for dynamic irrigation decision-making in the Agricultural Automation System. Draw the supporting UML diagram and implement the solution.

Expected Pattern:Strategy Pattern

Concept:Encapsulates different algorithms and allows them to be exchanged at runtime.

Other Pattern Scenario

Command Pattern

Smart Food Management System

Scenario:A Smart Food Management System provides an online food ordering service.Customers can perform different actions:Place food order.Cancel order.Modify order.Track delivery.The system should store customer requests so that previous operations can be logged, cancelled, or executed later.The ordering system should allow different commands without directly coupling the customer interface with restaurant operations.Requirement:What problem does the Command Design Pattern solve? Explain how the Command Pattern can be applied in the Smart Food Management System for handling customer orders. Draw the supporting UML diagram and implement the solution.

Expected Pattern:Command Pattern

Concept:Encapsulates a request as an object, allowing operations to be stored, queued, logged, or undone.

Other Pattern Scenario

State Pattern
Smart Traffic Management System (DUMS Module)
Scenario:A Smart Traffic Management System controls traffic signals at different road intersections.A traffic signal changes its behavior depending on its current state:Green State → Allow vehicles to move.Yellow State → Prepare vehicles to stop.Red State → Stop vehicles.The behavior of the traffic light should change automatically when its internal state changes without using complex conditional statements.Requirement:What problem does the State Design Pattern solve? Explain how the State Pattern can be used to model traffic signal behavior in the Smart Traffic Management System. Draw the supporting UML diagram and implement the solution.

Expected Pattern:State Pattern

Concept:Allows an object to change its behavior when its internal state changes.

Other Pattern Scenario

Adapter Pattern

Agricultural Automation System

Scenario:A Smart Agricultural Automation System uses an existing irrigation controller that accepts data from a standard soil moisture sensor.A newly purchased IoT soil sensor provides moisture information using a different interface. The existing irrigation controller cannot directly communicate with this new sensor.

Requirement:What problem does the Adapter Design Pattern solve? Explain how the Adapter Pattern can integrate the new IoT soil sensor without modifying the existing irrigation controller. Draw the supporting UML diagram and implement the solution.

Sample Question

What problem does Adapter design pattern solve? Explain with an example and draw supporting UML diagram.
Suppose You are designing a system for money withdrawal from bank account using check with the following authorization scenario:

- For Tk. < 10,000 only cashier's authorization is sufficient to withdraw money from account.
- For Tk. 10,000 to 10,00,000 authorization from both cashier and senior officer is needed.
- For Tk > 10,00,000 authorization from Senior officer and Manager is needed.

Implement this behaviour using appropriate software design pattern.

Sample Quesiton

Bloom's Level	Question
Remember	1. What is a behavioural design pattern? Name five behavioural patterns discussed in Module 2.
Understand	2. Explain the main purpose of the Template Method Pattern.
Understand	3. Explain how the Chain of Responsibility Pattern processes a request through multiple handlers.
Apply	4. Design a university complaint-handling process using the Chain of Responsibility Pattern, where complaints may be handled by a Class Teacher, Department Chairman, Dean, or Vice Chancellor.
Analyze	5. Analyze a program containing a large number of if-else statements to determine an object's behavior based on its current state. Explain how the State Pattern could improve the design.
Evaluate	6. Evaluate the advantages and disadvantages of using the State Pattern instead of a large switch statement.
Create	7. Design a Java-based online order system using the State Pattern with Pending, Paid, Shipped, and Delivered states.

Bloom's Level	Question
Remember	1. Define the Command Pattern and identify its main components.
Understand	2. Explain the purpose of the Command Pattern and how it encapsulates a request as an object.
Understand	3. Explain how the Command Pattern can support undo and redo operations.
Apply	4. Apply the Command Pattern to a text editor that supports Copy, Paste, Delete, and Undo operations. Identify the Command, Invoker, and Receiver.
Analyze	5. Analyze how the Command Pattern reduces coupling between a graphical user-interface button and the object that performs the actual operation.
Evaluate	6. Evaluate the benefits of using the Command Pattern for queueing, logging, scheduling, and undo/redo operations.
Create	7. Design a Java-based remote-control system using the Command Pattern where different buttons can execute different commands such as LightOn, LightOff, and FanOn.

Bloom's Level	Question
Remember	1. Define the Command Pattern and identify its main components.
Understand	2. Explain the purpose of the Command Pattern and how it encapsulates a request as an object.
Understand	3. Explain how the Command Pattern can support undo and redo operations.
Apply	4. Apply the Command Pattern to a text editor that supports Copy, Paste, Delete, and Undo operations. Identify the Command, Invoker, and Receiver.
Analyze	5. Analyze how the Command Pattern reduces coupling between a graphical user-interface button and the object that performs the actual operation.
Evaluate	6. Evaluate the benefits of using the Command Pattern for queueing, logging, scheduling, and undo/redo operations.
Create	7. Design a Java-based remote-control system using the Command Pattern where different buttons can execute different commands such as LightOn, LightOff, and FanOn.

Sample Quesiton

Bloom's Level	Question
Remember	1. Name five behavioural design patterns discussed in Module 2.
Understand	2. Explain the main idea of the Template Method Pattern.
Understand	3. Explain why the Chain of Responsibility Pattern reduces coupling between a request sender and request receiver.
Apply	4. A university has several levels of approval: Teacher → Chairman → Dean → Vice Chancellor. Apply the Chain of Responsibility Pattern to this approval system.
Analyze	5. Analyze a program where an object's behavior is controlled by a large switch statement based on its current state. Identify the problem and explain how the State Pattern could improve it.
Evaluate	6. Evaluate the advantages and disadvantages of using the Chain of Responsibility Pattern when many handlers are present in the chain.
Create	7. Design a Java-based customer-support system using the Chain of Responsibility Pattern with different support levels such as Level 1, Level 2, and Level 3 support.

Bloom's Level	Question
Remember	1. What is the Command Pattern? Name its major participants.
Understand	2. Explain how the Command Pattern encapsulates a request as an object.
Understand	3. Explain how the Command Pattern can support command queueing and scheduling.
Apply	4. Apply the Command Pattern to a text editor that provides Copy, Paste, Delete, and Undo operations. Identify the Client, Command, Invoker, and Receiver.
Analyze	5. Analyze a GUI application in which every button directly contains the application logic for performing its operation. Explain the coupling problem and how the Command Pattern can solve it.
Evaluate	6. Evaluate whether the Command Pattern is appropriate for implementing an undo/redo mechanism. Explain your reasoning.
Create	7. Create a Java-based restaurant ordering system using the Command Pattern where orders can be created, queued, executed, and cancelled.

Bloom's Level	Question
Remember	1. Define the Observer Pattern and identify the Subject and Observer.
Understand	2. Explain the one-to-many relationship used by the Observer Pattern.
Understand	3. Explain how the Observer Pattern reduces direct dependency between a subject and its dependent objects.
Apply	4. Apply the Observer Pattern to an online news system where multiple users receive notifications whenever a new article is published.
Analyze	5. Analyze a weather-monitoring system where the weather station directly calls update methods on every display component. Explain the design problems and suggest how the Observer Pattern could improve the system.
Evaluate	6. Compare the Observer Pattern with manually notifying every dependent object. Evaluate the advantages of using the Observer Pattern.
Create	7. Design a Java-based stock-price monitoring system using the Observer Pattern. Include a StockMarket Subject and at least three different Observer classes.